

Lab7

ส่วนที่ 1: Software Design & Principles Explanation

1.1 สถาปัตยกรรมของระบบและ GRASP Patterns

ระบบ Game Catalog ได้รับการออกแบบในลักษณะสถาปัตยกรรมแบบแบ่งชั้น (Layered Architecture) โดยมีการกำหนดหน้าที่ความรับผิดชอบของแต่ละคลาสตามหลักการ GRASP (General Responsibility Assignment Software Patterns) ดังนี้

1. Controller Pattern: คลาส GameController ทำหน้าที่เป็นจุดรับคำร้องขอ (HTTP Request) จากผู้ใช้งานผ่านเว็บเบราว์เซอร์แต่เพียงอย่างเดียว โดยไม่มีการฝังตรรกะทางธุรกิจ (Business Logic) หรือการเข้าถึงฐานข้อมูลไว้ในชั้นนี้ ทุกคำสั่งจะถูกส่งต่อไปยังชั้น Service เพื่อดำเนินการ ซึ่งสอดคล้องกับหลักการของ Controller Pattern ที่กำหนดให้มีตัวกลางรับเหตุการณ์จากระบบ (System Event) จากส่วนติดต่อผู้ใช้ แล้วจึงกระจายงานต่อไปยังชั้นถัดไป แทนที่จะให้ชั้นการแสดงผลติดต่อกับตรรกะทางธุรกิจโดยตรง
2. Information Expert: คลาส GameRepository ได้รับมอบหมายให้เป็นผู้เชี่ยวชาญด้านการเข้าถึงข้อมูล เนื่องจากเป็นคลาสที่มีความรู้ในการติดต่อกับตาราง games ในฐานข้อมูลโดยตรงผ่าน Spring Data JPA ในขณะที่คลาสภายใต้ Strategy Package แต่ละคลาสเป็นผู้เชี่ยวชาญด้านการคำนวณส่วนลด เนื่องจากแต่ละคลาสมีความรู้เฉพาะเกี่ยวกับสูตรการคำนวณของตนเอง
3. High Cohesion: แต่ละคลาสในระบบมีความรับผิดชอบเพียงหนึ่งประการที่ชัดเจน อาทิ NoDiscountStrategy มีหน้าที่คำนวณราคาโดยไม่มีส่วนลดเท่านั้น และ GameRepository มีหน้าที่ดำเนินการ CRUD กับฐานข้อมูลเท่านั้น โดยไม่มีการก๊อปปี้ข้อมูลอื่นปะปนอยู่ในคลาสเดียวกัน
4. Low Coupling: คลาส GameController ไม่มีการอ้างอิงถึง GameRepository โดยตรง แต่ติดต่อผ่านชั้น GameService เท่านั้น เช่นเดียวกับที่ DiscountContext ไม่มีการอ้างอิงถึงคลาสรูปธรรม (Concrete Class) ของแต่ละกลยุทธ์ แต่อ้างอิงผ่านอินเทอร์เฟซ DiscountStrategy เท่านั้น ส่งผลให้แต่ละชั้นสามารถเปลี่ยนแปลงการทำงานภายในได้โดยไม่ส่งผลกระทบต่อชั้นอื่น
5. Indirection: คลาส DiscountContext ทำหน้าที่เป็นตัวกลางเชื่อมโยง (Indirection) ระหว่างชั้น GameService กับกลยุทธ์การคำนวณส่วนลดที่แท้จริง ทำให้ GameService ไม่จำเป็นต้องรับรู้ถึงรายละเอียดของแต่ละกลยุทธ์ เพียงเรียกใช้เมธอด applyDiscount() ของ DiscountContext เท่านั้น

1.2 หลักการ SOLID ในระดับสูง

S - Single Responsibility Principle (SRP): แต่ละคลาสในระบบมีเหตุผลเพียงหนึ่งเดียวที่จะเปลี่ยนแปลง กล่าวคือ คลาส Game จะเปลี่ยนแปลงเมื่อโครงสร้างข้อมูลของเกมเปลี่ยนแปลง คลาส GameService จะเปลี่ยนแปลงเมื่อตรรกะทางธุรกิจเปลี่ยนแปลง คลาส GameController จะเปลี่ยนแปลงเมื่อการไหลของกระบวนการทำงานของเว็บแอปพลิเคชันเปลี่ยนแปลง และคลาสภายใต้ Strategy Package แต่ละคลาสจะเปลี่ยนแปลงเมื่อสูตรการคำนวณส่วนลดของตนเองเปลี่ยนแปลง

O - Open/Closed Principle (OCP): หลักการนี้ปรากฏชัดเจนที่สุดในคลาส DiscountContext กล่าวคือ หากมีความประสงค์จะเพิ่มประเภทส่วนลดใหม่ในอนาคต (ตัวอย่างเช่น HolidayDiscountStrategy) ผู้พัฒนาเพียงสร้างคลาสใหม่ที่ implement อินเทอร์เฟซ DiscountStrategy และกำหนดหมายเหตุ @Component พร้อมชื่อประจำตัว โดยไม่จำเป็นต้องแก้ไขโค้ดในคลาส DiscountContext แม้แต่บรรทัดเดียว เนื่องจาก Spring Framework จะทำการประกอบ (Inject) ตัวแปรชนิด Map<String, DiscountStrategy> ให้โดยอัตโนมัติ ระบบจึงมีคุณสมบัติ "เปิดให้ขยายความสามารถ แต่ปิดการแก้ไขโค้ดเดิม" ตามหลักการ OCP อย่างสมบูรณ์

L - Liskov Substitution Principle (LSP): คลาสที่ implement อินเทอร์เฟซ DiscountStrategy ทุกคลาส ได้แก่ NoDiscountStrategy, StudentDiscountStrategy และ SeasonalSaleStrategy สามารถใช้แทนกันได้ ในทุกบริบทที่มีการเรียกใช้งานผ่านอินเทอร์เฟซ โดยยังคงพฤติกรรมที่ถูกต้องตามสัญญาที่อินเทอร์เฟซกำหนดไว้ กล่าวคือ คำนวณราคาที่คำนวณแล้วและชื่อประเภทส่วนลด โดยไม่มีคลาสใดที่ก่อให้เกิดข้อผิดพลาดหรือพฤติกรรมที่ขัดแย้งกับสัญญาดังกล่าว

I - Interface Segregation Principle (ISP): อินเทอร์เฟซ DiscountStrategy ประกอบด้วยเมธอดเพียงสองรายการที่จำเป็นต่อการใช้งานจริง ได้แก่ calculateDiscount() และ getDisplayName() จึงไม่เป็นการให้คลาสที่นำไปใช้งานต้อง implement เมธอดที่ไม่เกี่ยวข้องกับหน้าที่ของตน

D - Dependency Inversion Principle (DIP): คลาส DiscountContext ซึ่งเป็นโมดูลระดับสูง (High-level Module) ไม่มีการพึ่งพาคลาสรูปธรรมของแต่ละกลยุทธ์ซึ่งเป็นโมดูลระดับล่าง (Low-level Module) โดยตรง แต่พึ่งพาด้านนามธรรม (Abstraction) ในรูปแบบอินเทอร์เฟซ DiscountStrategy โดยอาศัยการประกอบผ่านตัวสร้าง (Constructor Injection) เช่นเดียวกับที่ GameService พึ่งพา GameRepository และ DiscountContext ผ่านนามธรรมเท่านั้น โดยไม่มีการสร้างอินสแตนซ์ (Instantiation) ขึ้นด้วยคำสั่ง new ปรากฏอยู่ในระบบ

1.3 การประยุกต์ใช้ Strategy Pattern

ระบบได้ประยุกต์ใช้ Strategy Pattern ในการคำนวณส่วนลดราคาเกม โดยประกอบด้วยองค์ประกอบดังนี้

- อินเทอร์เฟซ DiscountStrategy กำหนดสัญญาว่ากลยุทธ์ทุกประเภทต้องมีเมธอด calculateDiscount(price) และ getDisplayName()
- คลาสรูปธรรม NoDiscountStrategy, StudentDiscountStrategy (ให้ส่วนลดร้อยละ 10) และ SeasonalSaleStrategy (ให้ส่วนลดร้อยละ 20) ซึ่งแต่ละคลาสแทนอัลกอริทึมการคำนวณราคาที่แตกต่างกัน
- คลาส DiscountContext ทำหน้าที่เป็นตัวกลาง โดยรับตัวแปรชนิด Map<String, DiscountStrategy> ผ่าน Constructor Injection จาก Spring Framework แล้วเลือกกลยุทธ์ที่สอดคล้องกับค่า discountType ของเกม นั้นมาใช้งานขณะระบบทำงานจริง (Runtime)

ในด้านประโยชน์ที่สอดคล้องกับหลักการ Open/Closed Principle นั้น หากระบบใช้โครงสร้างเงื่อนไข (if-else หรือ switch-case) ในการคำนวณส่วนลดแบบดั้งเดิม การเพิ่มโปรโมชั่นใหม่จะจำเป็นต้องแก้ไขโค้ดเดิมทุกครั้ง ซึ่งมีความเสี่ยงต่อการก่อให้เกิดข้อผิดพลาดต่อฟังก์ชันที่มีอยู่เดิม ในทางตรงกันข้าม การประยุกต์ใช้ Strategy Pattern ทำให้การเพิ่มส่วนลดประเภทใหม่สามารถกระทำได้เพียงการเพิ่มคลาสใหม่ โดยไม่จำเป็นต้องแก้ไขโค้ดที่มีอยู่เดิมแต่อย่างใด ซึ่งสอดคล้องกับหลักการ Open/Closed Principle อย่างสมบูรณ์

1.4 สถาปัตยกรรมแบบแบ่งชั้น (Layered Architecture)

ระบบมีการแบ่งโครงสร้างออกเป็น 4 ชั้น ได้แก่ Controller, Service, Repository และ Database ตามลำดับ โดยเฉพาะอย่างยิ่งการแยกชั้น Service ออกจากชั้น Controller และ Repository ก่อให้เกิดประโยชน์ดังนี้

- **ด้าน Low Coupling:** ชั้น Controller ไม่มีความผูกพันกับวิธีการเข้าถึงข้อมูลโดยตรง หากมีการเปลี่ยนแปลงเทคโนโลยีการเข้าถึงข้อมูลในอนาคต การแก้ไขจะจำกัดอยู่เพียงชั้น Service และ Repository เท่านั้น โดยไม่ส่งผลกระทบต่อชั้น Controller
- **ด้าน High Cohesion:** ตรรกะทางธุรกิจ อาทิ การคำนวณส่วนลดผ่าน DiscountContext ถูกรวมศูนย์ไว้ที่ชั้น Service เพียงจุดเดียว โดยไม่กระจัดกระจายอยู่ทั้งในชั้น Controller และ Repository
- **ด้านความสามารถในการทดสอบ (Testability):** ผู้พัฒนาสามารถจำลอง (Mock) การทำงานของ GameRepository และ DiscountContext เพื่อทดสอบชั้น GameService ได้โดยไม่ต้องพึ่งพาฐานข้อมูลหรือเซิร์ฟเวอร์เว็บจริง

1.5 ลำดับการทำงานของระบบ (Execution Flow)

ตัวอย่างลำดับการทำงานในกรณีแสดงหน้ารายการเกม (คำร้องขอแบบ GET /games) มีดังนี้

1. เว็บเบราว์เซอร์ส่งคำร้องขอ HTTP แบบ GET ไปยังเส้นทาง /games
2. DispatcherServlet ของ Spring Framework ส่งต่อคำร้องขอไปยังเมธอด getGames() ของคลาส GameController
3. GameController เรียกใช้งานเมธอด getAllGames() ของ GameService โดยไม่จำเป็นต้องรับรู้รายละเอียดการทำงานภายใน
4. GameService เรียกใช้งานเมธอด findAll() ของ GameRepository เพื่อดึงข้อมูลเกมทั้งหมดจากตาราง games ในฐานข้อมูล PostgreSQL ผ่าน Spring Data JPA และ Hibernate
5. สำหรับข้อมูลเกมแต่ละรายการ GameService เรียกใช้งานเมธอด applyDiscount() ของ DiscountContext โดยส่งราคาและประเภทส่วนลดเข้าไป DiscountContext จะเลือกกลยุทธ์ที่สอดคล้องกับประเภทส่วนลดจากตัวแปร Map ที่ได้รับผ่านการประกอบขึ้น แล้วคืนค่าราคาสุทธิและชื่อประเภทส่วนลดกลับมา
6. GameService กำหนดค่าราคาสุทธิและชื่อประเภทส่วนลดลงในอินสแตนซ์ของ Game (ซึ่งเป็นฟิลด์ประเภท @Transient ที่ไม่มีการบันทึกลงฐานข้อมูล) แล้วส่งกลับไปยัง Controller
7. GameController บรรจุผลลัพธ์ลงใน Model แล้วส่งคืนชื่อวิว (View) "games/list"
8. Thymeleaf ทำการประมวลผลเทมเพลต list.html โดยดึงข้อมูลจาก Model มาแสดงผล แล้วส่งเอกสาร HTML กลับไปยังเว็บเบราว์เซอร์เพื่อแสดงผลแก่ผู้ใช้งาน

ส่วนที่ 2: โครงสร้างโค้ดและคำอธิบาย (Code Implementation & Explanation)

การประยุกต์ใช้ Dependency Injection แบบ Constructor Injection ในแต่ละชั้นของระบบสามารถสรุปได้ดังตารางต่อไปนี้

GameController รับผ่าน Constructor GameService

```
public GameController(GameService gameService) {
    this.gameService = gameService;
}
```

ไม่จำเป็นต้องสร้างอินสแตนซ์ของ GameService ด้วยตนเอง โดยมอบหมายให้ Spring Framework จัดการวงจรชีวิตของอ็อบเจกต์ (Object Lifecycle) ซึ่งเกี่ยวข้องการเปลี่ยนแปลงการทำงานภายในและการทดสอบระบบ

GameService รับผ่าน Constructor GameRepository, DiscountContext

```
public GameService(GameRepository gameRepository, DiscountContext discountContext) {
    this.gameRepository = gameRepository;
    this.discountContext = discountContext;
}
```

พึ่งพาด้านนามธรรม (Abstraction) ทั้งสองส่วน โดยไม่ผูกติดกับการทำงานเฉพาะเจาะจงใดๆ

DiscountContext รับผ่าน Constructor Map<String, DiscountStrategy>

```
public DiscountContext(Map<String, DiscountStrategy> strategies) {
    this.strategies = strategies;
}
```

Spring Framework ทำการรวบรวมทุก빈 (Bean) ที่ implement อินเทอร์เฟซ DiscountStrategy เข้าสู่ตัวแปร Map โดยอัตโนมัติ โดยใช้ชื่อประจำตัวของ빈 (เช่น @Component("student")) เป็นคีย์ ส่งผลให้การเพิ่มกลยุทธ์ใหม่ในอนาคตสามารถทำได้โดยไม่จำเป็นต้องแก้ไข Constructor ของ DiscountContext แต่อย่างใด

ส่วนที่ 3: Web Application & Database Screenshots

1. หน้าจอเพิ่มเกมใหม่ (Create)

Game Catalog CP353002 — Lab 7

← กลับไปรายการเกม

+ เพิ่มเกมใหม่
กรอกข้อมูลเกมที่ต้องการเพิ่มลงในระบบ

ชื่อเกม
Elden Ring (673380588-4 SEC 3)

แนวเกม (Genre)
Action RPG

แพลตฟอร์ม (Platform)
PC / PS5

คะแนน (Rating)
9.8

ราคาปกติ (บาท)
1790.00

% ส่วนลด (Strategy)
ส่วนลดนักศึกษา (10%)

วันวางจำหน่าย (Release Date)
02/25/2022

2. หน้าจอแสดงรายการเกมทั้งหมด (Read)

Game Catalog CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

รายการเกมทั้งหมด
จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete + เพิ่มเกมใหม่

✅ อัปเดตเกมสำเร็จ

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Elden Ring (673380588-4 SEC 3)	Action RPG	PC / PS5	★ 9.8	25/02/2022	ส่วนลดนักศึกษา 10%	1611.00 1790.00	
2	Expedition 33 (673380588-4 SEC 3)	RPG Turnbase	PC, PS5	★ 9.9	24/04/2025	ส่วนลดพิเศษ 20%	880.00 1100.00	

ทั้งหมด 2 เกม

CP353002 Principles of Software Design — Lab 7: Database Connectivity

3. หน้าจอแก้ไขเกม (Update)

Game Catalog CP353002 — Lab 7

← กลับไปรายการเกม

แก้ไขข้อมูลเกม
แก้ไขข้อมูลเกม: Expedition 33 (673380588-4 SEC 3)

ชื่อเกม
Expedition 33 (673380588-4 SEC 3)

แนวเกม (Genre)
RPG Turnbase

แพลตฟอร์ม (Platform)
PC, PS5

คะแนน (Rating)
9.9

ราคาปกติ (บาท)
1100.0

% ส่วนลด (Strategy)
ส่วนลดเทศกาล (20%)

วันวางจำหน่าย (Release Date)
02/25/2022

อัปเดต **ยกเลิก**

4. ผลลัพธ์การอัปเดตรายการเกม (โปรโมชัน)

Game Catalog CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

รายการเกมทั้งหมด
จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete ➕ เพิ่มเกมใหม่

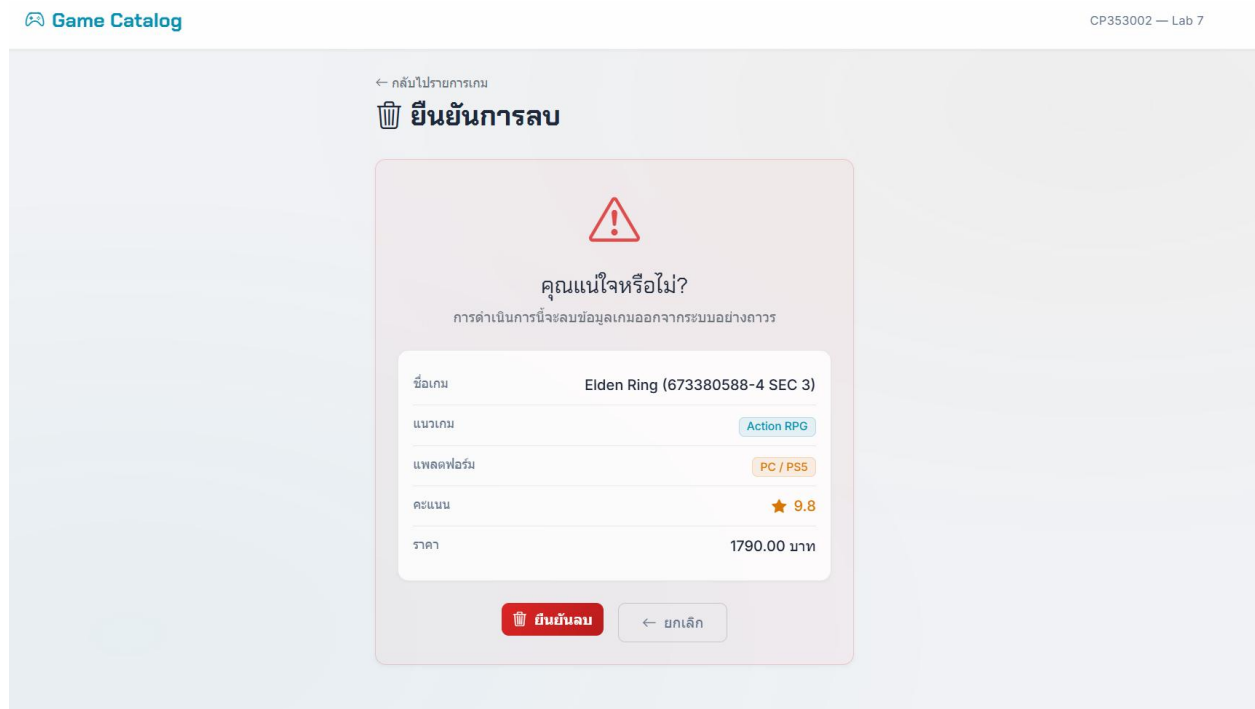
✅ อัปเดตเกมสำเร็จ

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Expedition 33 (673380588-4 SEC 3)	RPG Turnbase	PC, PS5	★ 9.9	24/04/2025	ส่วนลดเทศกาล 20%	880.00 1100.00	
2	Elden Ring (673380588-4 SEC 3)	Action RPG	PC / PS5	★ 9.8	25/02/2022	ส่วนลดเทศกาล 20%	1432.00 1790.00	

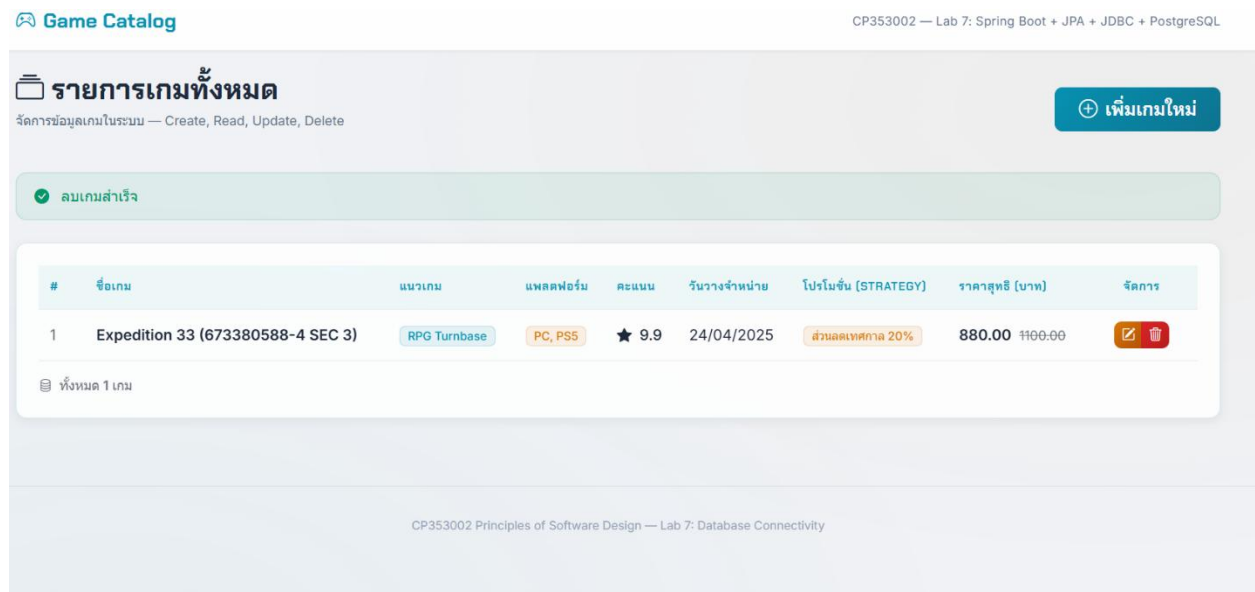
📖 ทั้งหมด 2 เกม

CP353002 Principles of Software Design — Lab 7: Database Connectivity

5. หน้าจอยืนยันลบ



6. ผลลัพธ์หลังลบ (Delete)



7. หน้าจอ Database

The screenshot displays the pgAdmin 4 web interface. On the left, the 'Object Explorer' shows the database structure, including schemas, tables, and columns. The 'games' table is selected, showing its columns: id, discount_type, genre, platform, price, rating, release_date, and title. The main pane shows a SQL query: `SELECT * FROM games;`. Below the query, the 'Data Output' tab displays the query results in a table format.

id	discount_type	genre	platform	price	rating	release_date	title
1	SEASONAL	RPG Turnbase	PC, PS5	1100	9.9	2025-04-24	Expedition 33 (673380588-4 SEC...

At the bottom, the status bar indicates 'Total rows: 1' and 'Query complete 00:00:00.167'.